ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 15

HTTPS + App Shell

На этом занятии обеспечим безопасность приложения с помощью локального HTTPS и реализуем архитектуру App Shell для мгновенной загрузки интерфейса. Рассмотрим настройки доверенного соединения на локальной машине и проектирование PWA так, чтобы пользователь видел каркас приложения практически мгновенно.

Введение

`HTTPS` - это протокол защищенной передачи данных, расширение протокола HTTP. Его задача — защитить информацию, которую отправляют и получают на сайте: логины, пароли, данные банковской карты.

Браузеры требуют HTTPS для работы многих современных возможностей: Service Worker, геолокация, уведомления и др. Если ваше приложение работает по HTTP, Service Worker не зарегистрируется, и все преимущества PWA будут недоступны. На локальном компьютере мы можем создать самоподписанный сертификат и доверить ему, чтобы эмулировать безопасное соединение.

`App Shell` (каркас приложения) - это минимальный набор HTML, CSS и JavaScript, который обеспечивает базовую структуру интерфейса (шапка, меню, основной контейнер). Этот каркас кэшируется при первом посещении и затем грузится мгновенно даже при плохом соединении. Внутрь контейнера динамически подгружается контент. Такой подход даёт ощущение нативной производительности.

Шаг 1. Настройка локального HTTPS

Чтобы использовать Service Worker и другие возможности PWA, нам нужно запустить приложение по HTTPS даже на локальной машине. Для этого удобно использовать утилиту `mkcert`.

1.1 Установка mkcert

- **Windows** (через Chocolatey):

```
choco install mkcert
```

- **macOS** (через Homebrew):

```
brew install mkcert
```

- **Linux**:

```
sudo apt install libnss3-tools
curl -JLO "https://dl.filippo.io/mkcert/latest?for=linux/amd64"
chmod +x mkcert-v*-linux-amd64
sudo mv mkcert-v*-linux-amd64 /usr/local/bin/mkcert
```

1.2 Генерация сертификатов

Выполните в корне вашего проекта:

```
mkcert -install
mkcert localhost 127.0.0.1 ::1
```

После этого в папке появятся два файла: `localhost.pem` (сертификат) и `localhost-key.pem` (ключ). Они понадобятся для запуска HTTPS-сервера.

1.3 Запуск сервера с HTTPS

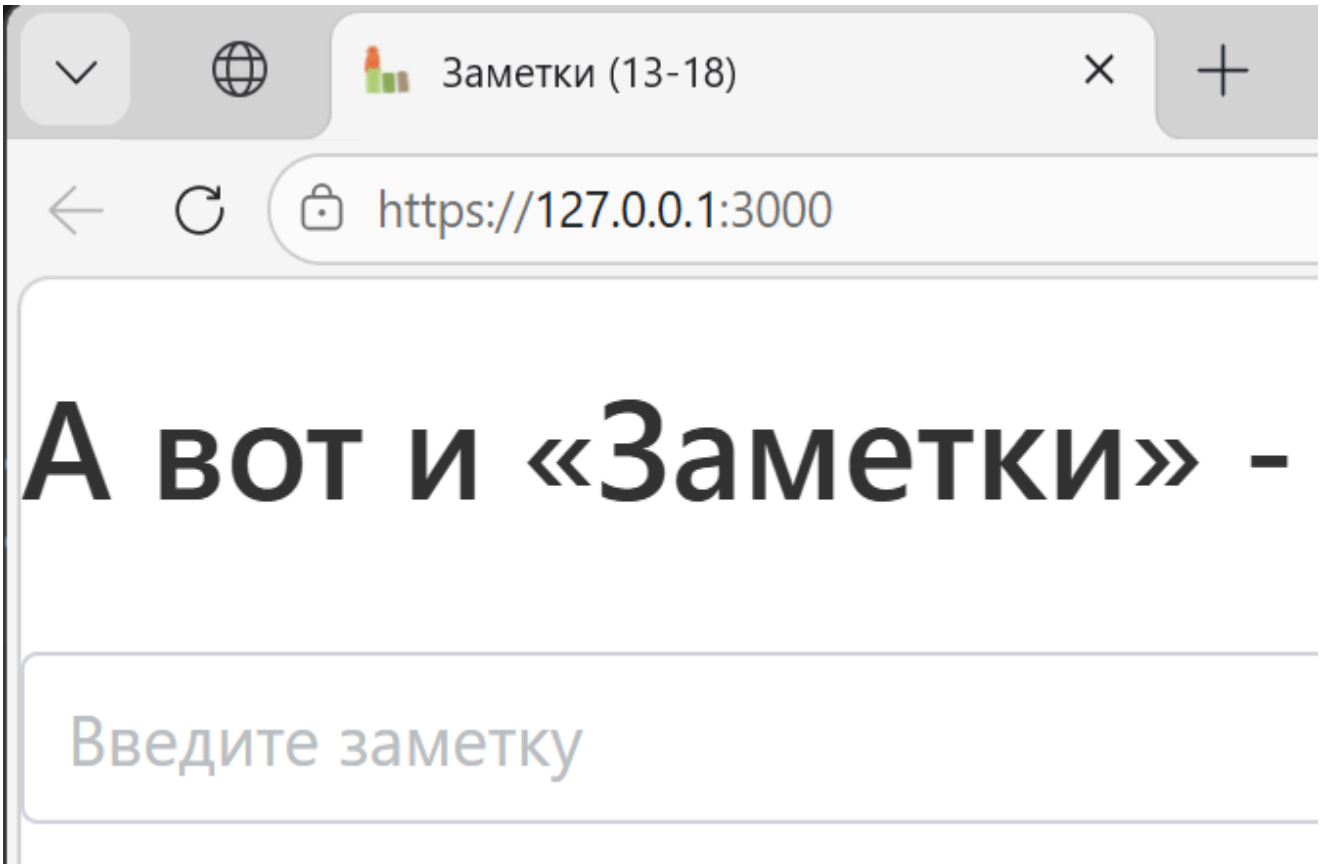
Установите `http-server` глобально (если ещё не сделали):

```
npm install -g http-server
```

Затем запустите сервер, указав ключи (**обратите внимание на их названия и, при необходимости, поправьте команду**):

```
http-server --ssl --cert localhost.pem --key localhost-key.pem -p 3000
```

Откройте в браузере адрес `https://localhost:3000`. Если всё настроено правильно, вы увидите замочек в адресной строке, что представлено на Рисунке ниже. В DevTools на вкладке **Security** должен отображаться статус «Secure».



Шаг 2. Реализация App Shell для приложения заметок

Теперь модернизируем наше приложение для заметок (из практических занятий 13–14) так, чтобы оно использовало архитектуру App Shell.

2.1 Структура проекта

Поправим структуру проекта согласно формату, представленному ниже.

```
notes-app/
├── content/
│   ├── home.html
│   └── about.html
├── icons/
│   ├── .......
│   ├── icon-192.png
│   ├── icon-512.png
│   └── favicon.ico
├── index.html
├── app.js
├── manifest.json
└── sw.js
```

2.2 index.html (каркас приложения)

```html
<!DOCTYPE html>
<html lang="ru">
```

```html
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="manifest" href="/manifest.json">
    <meta name="mobile-web-app-capable" content="yes">
    <meta name="theme-color" content="#4285f4">
    <link rel="apple-touch-icon" href="/icons/icon-152x152.png">
    <meta name="apple-mobile-web-app-status-bar-style" content="black-
translucent">
    <title>Заметки (13-18)</title>
    <link rel="stylesheet" href="https://unpkg.com/chota@latest">
    <link rel="icon" href="icons/favicon.ico" sizes="16x16"
type="image/ico">
</head>

<body>
    <header>
        <h1>А вот и «Заметки» - наше первое <span class="no-break">оффлайн-
приложение</span></h1>
        <nav class="tabs is-center">
            <button id="home-btn" class="tab active col-6">Главная</button>
            <button id="about-btn" class="tab col-6">О приложении</button>
        </nav>
    </header>

    <!-- Основной контейнер для динамического контента -->
    <main id="app-content" class="container" style="margin-top: 2rem;">
        <!-- Сюда будет загружаться контент -->
    </main>

    <script src="app.js"></script>
</body>

</html>
```

2.3 content/home.html (динамический контент главной страницы)

Здесь расположена форма добавления заметок и список заметок. Именно этот фрагмент будет подгружаться при выборе пункта «Главная».

```html
<div class="home-content">
    <h2 class="is-center">Добавить заметку</h2>
    <form id="note-form" class="row is-center">
        <input class="col-9" type="text" id="note-input"
placeholder="Введите текст заметки" required>
        <button class="col-3 button primary" type="submit">Добавить</button>
    </form>
```

```
    <h2 class="is-center" style="margin-top: 2rem;">Список заметок</h2>
    <ul id="notes-list" style="list-style: none; padding-left: 0;"></ul>
</div>
```

2.4 content/about.html

Страница «О приложении» с краткой информацией.

```
<div class="about-content">
    <h2 class="is-center">О приложении</h2>
    <p class="is-center">Версия 1.2.3</p>
    <p>Это приложение для заметок</p>
    <p>Оно классное и многое умеет</p>
</div>
```

2.5 app.js (основная логика)

В этом файле реализована навигация, загрузка контента через `fetch`, а также функционал заметок (сохранение в `localStorage`). Обратите внимание, что при загрузке страницы «Главная» вызывается `initNotes()` для инициализации формы и списка.

```
const contentDiv = document.getElementById('app-content');
const homeBtn = document.getElementById('home-btn');
const aboutBtn = document.getElementById('about-btn');

function setActiveButton(activeId) {
    [homeBtn, aboutBtn].forEach(btn => btn.classList.remove('active'));
    document.getElementById(activeId).classList.add('active');
}

async function loadContent(page) {
    try {
        const response = await fetch(`/content/${page}.html`);
        const html = await response.text();
        contentDiv.innerHTML = html;

// Если загружена главная страница, инициализируем функционал заметок
        if (page === 'home') {
            initNotes();
        }
    } catch (err) {
        contentDiv.innerHTML = `<p class="is-center text-error">Ошибка
загрузки страницы.</p>`;
        console.error(err);
    }
}
```

```
homeBtn.addEventListener('click', () => {
    setActiveButton('home-btn');
    loadContent('home');
});

aboutBtn.addEventListener('click', () => {
    setActiveButton('about-btn');
    loadContent('about');
});

// Загружаем главную страницу при старте
loadContent('home');

// Функционал заметок (localStorage)
function initNotes() {
    const form = document.getElementById('note-form');
    const input = document.getElementById('note-input');
    const list = document.getElementById('notes-list');

    function loadNotes() {
        const notes = JSON.parse(localStorage.getItem('notes') || '[]');
        list.innerHTML = notes.map(note => `<li class="card" style="margin-
bottom: 0.5rem; padding: 0.5rem;">${note}</li>`).join('');
    }

    function addNote(text) {
        const notes = JSON.parse(localStorage.getItem('notes') || '[]');
        notes.push(text);
        localStorage.setItem('notes', JSON.stringify(notes));
        loadNotes();
    }

    form.addEventListener('submit', (e) => {
        e.preventDefault();
        const text = input.value.trim();
        if (text) {
            addNote(text);
            input.value = '';
        }
    });

    loadNotes();
}

// Регистрация Service Worker
if ('serviceWorker' in navigator) {
    window.addEventListener('load', () => {
        navigator.serviceWorker.register('/sw.js')
            .then(reg => console.log('SW registered:', reg.scope))
            .catch(err => console.log('SW registration failed:', err));
```

```
    });
}
```

2.6 sw.js (Service Worker с поддержкой App Shell)

Этот файл содержит логику кэширования. Статические ресурсы (App Shell) кэшируются при установке (стратегия Cache First). Динамические страницы (`/content/*`) загружаются сначала из сети, а при недоступности сети возвращаются из кэша (с фолбеком на `home.html`).

```
const CACHE_NAME = 'notes-cache-v2';
const DYNAMIC_CACHE_NAME = 'dynamic-content-v1';
const ASSETS = [
    '/',
    '/index.html',
    '/app.js',
    '/manifest.json',
    '/icons/favicon-16x16.png',
    '/icons/favicon-32x32.png',
    '/icons/favicon-48x48.png',
    '/icons/favicon-64x64.png',
    '/icons/favicon-128x128.png',
    '/icons/favicon-256x256.png',
    '/icons/favicon-512x512.png'
];

self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => cache.addAll(ASSETS))
            .then(() => self.skipWaiting())
    );
});

self.addEventListener('activate', event => {
    event.waitUntil(
        caches.keys().then(keys => {
            return Promise.all(
                keys.filter(key => key !== CACHE_NAME && key !==
DYNAMIC_CACHE_NAME)
                    .map(key => caches.delete(key))
            );
        }).then(() => self.clients.claim())
    );
});

// Для статики - Cache First, для контента - Network First
self.addEventListener('fetch', event => {
```

```
  const url = new URL(event.request.url);

  // Пропускаем запросы к другим источникам (например, к CDN chota)
  if (url.origin !== location.origin) return;

  // Динамические страницы (content/*) - сначала сеть, затем кэш
  if (url.pathname.startsWith('/content/')) {
    event.respondWith(
      fetch(event.request)
        .then(networkRes => {
          // Кэшируем свежий ответ
          const resClone = networkRes.clone();
          caches.open(DYNAMIC_CACHE_NAME).then(cache => {
            cache.put(event.request, resClone);
          });
          return networkRes;
        })

        .catch(() => {
          // Если сеть недоступна, берём из кэша (или home как fallback)
          return caches.match(event.request)
            .then(cached => cached || caches.match('/content/home.html'));
        })
    );
  }
});
```

Шаг 3. Проверка работы

1. Запустите сервер с HTTPS.
2. В DevTools на вкладке «Application» -> «Service Workers» убедитесь, что Service Worker активирован.
3. Перейдите на вкладку «Cache Storage» - вы должны увидеть два кэша: `app-shell-v2` и `dynamic-content-v1`. В первом лежат статические файлы.
4. Добавьте несколько заметок - они сохранятся в `localStorage`.
5. На вкладке «Network» установите ограничение скорости (например, Slow 3G) и перезагрузите страницу. Вы увидите, что каркас (шапка, меню) отображается сразу, а контент подгружается чуть позже.
6. Отключите сеть и обновите страницу. Приложение должно полностью загрузиться из кэша, а заметки останутся на месте. Вы даже сможете добавлять новые заметки офлайн – они будут сохранены локально.

Практическое задание

Доработайте реализованное ранее приложение для заметок с импользованием архитектуру App Shell. Приложение должно работать по HTTPS. Проверьте, что при первом посещении кэшируются все статические ресурсы, а динамические страницы

загружаются по стратегии Network First. Добавьте страницу «О нас» с информацией о приложении. Убедитесь, что новая страница также корректно кэшируются и доступны офлайн.

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.